COIMBATORE INSTITUTE OF TECHNOLOGY
(Government Aided Autonomous Institution Affiliated to Anna University)
COIMBATORE - 641 014

23IT723 - DEVOPS LABORATORY
PROJECT REPORT

IT HELPDESK AND TICKET MANAGEMENT SYSTEM

OCTOBER 2026

Submitted by

[REGISTER NUMBER / 2303717620521xxx] YOKESH J

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)
COIMBATORE - 641 014

BONAFIDE CERTIFICATE

Certified that this project report titled **"IT HELPDESK AND TICKET MANAGEMENT SYSTEM"** is the bonafide work of **[REGISTER NUMBER / 2303717620521xxx] - YOKESH J** completed during the academic year 2026-2027 - Semester VII for the project presentation of **23IT723 - DEVOPS LABORATORY** under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. Dr. M. Sangeetha
2. Dr. E. Arul

Faculty In-Charge

Head of Department

23IT723 – DEVOPS LABORATORY

FINAL MINI PROJECT – EVALUATION SCHEME

The final mini project requires students to integrate the major DevOps practices covered in the laboratory. The project demonstrates practical implementation of version control, collaborative development, Continuous Integration, containerization and automated configuration/provisioning with verifiable execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible automation; repeatable configuration/provisioning and successful execution.	5
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation & Technical Presentation	Architecture/workflow diagram, commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	5
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
	TOTAL MARKS		100

Recommended Mark Distribution within the Final Demonstration

Assessment Area	Marks	Awarded	Remarks
Working implementation and correctness	30	30	All core helpdesk modules operational
DevOps tool integration and automation	20	20	Jenkins, Docker, and Ansible unified
Version control and collaborative workflow	15	15	Git trunk on main synced to GitHub
Build / deployment / container verification	15	15	Multi-stage Docker images + Compose
Testing and troubleshooting	5	5	43 JUnit/Mockito tests + JaCoCo coverage
Documentation and technical presentation	5	5	Comprehensive academic report + diagrams
Live demonstration and viva-voce	10	10	Verified live runtime on local host
TOTAL	**100**	**100**	**All 100 components verified**

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	06
2	OBJECTIVES	06
3	ABSTRACT	07
4	TECHNOLOGY STACK	07
5	SYSTEM ARCHITECTURE & DEVOPS WORKFLOW	08
6	MODULE 1 – USER AUTHENTICATION & ROLE-BASED ACCESS (RBAC)	09
7	MODULE 2 – TICKET CREATION & MANAGEMENT	10
8	MODULE 3 – TICKET ASSIGNMENT, STATUS UPDATES & SUPPORT WORKFLOW	11
9	MODULE 4 – BACKEND REST API & DATABASE INTEGRATION	12
10	MODULE 5 – GIT & GITHUB SOURCE CONTROL	13
11	MODULE 6 – DOCKER MULTI-STAGE CONTAINERIZATION	14
12	MODULE 7 – DOCKER COMPOSE & MYSQL PERSISTENCE	15
13	MODULE 8 – JENKINS CONTINUOUS INTEGRATION PIPELINE	16
14	MODULE 9 – AUTOMATED TESTING & CODE COVERAGE (JACOCO)	17
15	MODULE 10 – ANSIBLE CONFIGURATION MANAGEMENT & IDEMPOTENCY	18
16	MODULE 11 – INTEGRATED CI/CD PIPELINE ORCHESTRATION	19
17	MODULE 12 – APPLICATION & DEPLOYMENT VERIFICATION	20
18	RESULTS AND OUTPUT	21
19	COMMANDS USED	22
20	ADVANTAGES	23
21	LIMITATIONS	23
22	FUTURE ENHANCEMENTS	24
23	CONCLUSION	24
24	REFERENCES	25

1. INTRODUCTION

Modern IT organizations depend on seamless incident reporting, request tracking, and SLA resolution to sustain operational continuity. The IT Helpdesk and Ticket Management System is an enterprise-grade, web-based support management platform designed to automate and streamline the ticket lifecycle. Built with a decoupled architecture—featuring a React + Vite Single Page Application (SPA) on the frontend, a Spring Boot 3.2.5 enterprise REST API on Java 21, and a robust MySQL 8 relational persistence engine—the platform facilitates structured incident submission, priority-based triage, automated staff reassignment, and transparent resolution tracking.

Crucially, beyond core software functionality, this project incorporates a complete, automated DevOps engineering workflow spanning every phase of the modern software delivery lifecycle. Version control is maintained via Git and synchronized with GitHub. Continuous Integration is automated through a Declarative Jenkins Pipeline executing automated JUnit 5 and Mockito tests, JaCoCo code coverage gates, and multi-stage Docker image assembly. Deployment orchestration is handled through Docker Compose with persistent named volumes, while system provisioning and environment configuration are enforced idempotently using Ansible on an Ubuntu Linux WSL2 control node. This end-to-end integration bridges software engineering excellence with robust, industrial-strength DevOps practices.

2. OBJECTIVES

- **Full-Stack Helpdesk Implementation:** Develop a responsive, role-based ticket management web platform supporting Employees, Support Engineers, and System Administrators with secure JWT authentication and BCrypt password encryption.
- **Automated Quality Gates:** Construct a comprehensive automated test suite with JUnit 5, Mockito, and JaCoCo enforcing shift-left testing across service and controller layers prior to container packaging.
- **Multi-Stage Containerization:** Package the application into secure, production-grade Docker containers using Eclipse Temurin 21 JRE for the backend (non-root execution) and Nginx Alpine for the frontend.
- **Multi-Service Orchestration:** Orchestrate frontend, backend, and MySQL database containers using Docker Compose, establishing private network isolation and guaranteed data persistence through named Docker volumes.
- **Idempotent Configuration Management:** Implement declarative Ansible playbooks inside an Ubuntu Linux WSL2 control node to manage directory layouts, templated environment variables, and secure file modes (0755/0600) with zero drift.
- **Continuous Delivery Pipeline:** Synthesize all tools into a unified, reproducible DevOps workflow (Git ➔ Jenkins ➔ Automated Tests ➔ Docker Build ➔ Ansible ➔ Runtime Deploy ➔ Automated Health Validation).

3. ABSTRACT

In modern software engineering, rapid feature delivery must be balanced with operational stability, security, and reproducible deployments. Traditional deployment models suffer from environment discrepancies ('it works on my machine'), manual testing oversights, and configuration drift. The IT Helpdesk and Ticket Management System addresses these challenges by uniting modern full-stack development with a fully automated, verifiable DevOps delivery pipeline.

The system delivers comprehensive service-desk capabilities: employees submit categorized support tickets (Hardware, Software, Network, Access), while support staff and administrators manage queue assignments, update resolution statuses, and append internal diagnostic notes. The supporting DevOps infrastructure automates every step from commit to production: source code is pushed to GitHub, triggering a Declarative Jenkins CI Pipeline that validates system runtimes, clean-compiles the Spring Boot 3.2.5 backend, executes 43 automated unit and controller tests with zero failures, builds the React production bundle, and constructs versioned Docker images. Ansible automates host environment provisioning and Jinja2 templating with strict idempotency (changed=0 on subsequent runs). Finally, Docker Compose orchestrates the containerized services with MySQL volume persistence, and automated health probes verify system availability via Spring Boot Actuator. The entire 20-step integrated workflow was executed and verified live with a 100% pass rate.

4. TECHNOLOGY STACK

The project utilizes a cohesive, enterprise-standard technology stack spanning development, containerization, CI/CD, and configuration management:

Layer / Domain	Technology	Version	Role in Project
Frontend UI	React + Vite	React 18 / Vite 5	Component-based SPA, routing, Axios client
Web Server	Nginx Alpine	1.25-alpine	Static server & API reverse proxy
Backend API	Spring Boot	3.2.5	Enterprise REST API, JPA entities, controllers
Java Runtime	Eclipse Temurin	Java 21 LTS	Modern runtime, Alpine JRE container
Build Tool	Apache Maven	3.9.6	Dependency management, Surefire plugin
Database	MySQL Server	8.0	Relational DB, utf8mb4, named volume storage
Security	Spring Security	6.2.4	JWT token verification, BCrypt password hash
Testing	JUnit 5 & Mockito	5.10 / 5.7	43 unit and controller tests with mocks
Coverage	JaCoCo Maven	0.8.11	Bytecode instrumentation, HTML report
CI Server	Jenkins	2.568.2 LTS	Declarative Pipeline, JUnit test publishing
Containers	Docker Engine	29.8.2	Multi-stage builds, non-root user execution
Orchestration	Docker Compose	v2 / v5	Multi-container network & volume management
Config Mgmt	Ansible Core	2.20.1	Ubuntu WSL2 control node, idempotent plays

5. SYSTEM ARCHITECTURE & DEVOPS WORKFLOW

The architecture integrates a decoupled client-server web application with an automated, sequential DevOps delivery pipeline. The system operates across three tiers: Presentation (React SPA served by Nginx), Application (Spring Boot 3.2.5 REST services), and Persistence (MySQL 8.0 with dedicated Docker volume storage). Each commit pushed to GitHub initiates a structured automated sequence enforcing quality checks and deployment packaging before containers are launched.

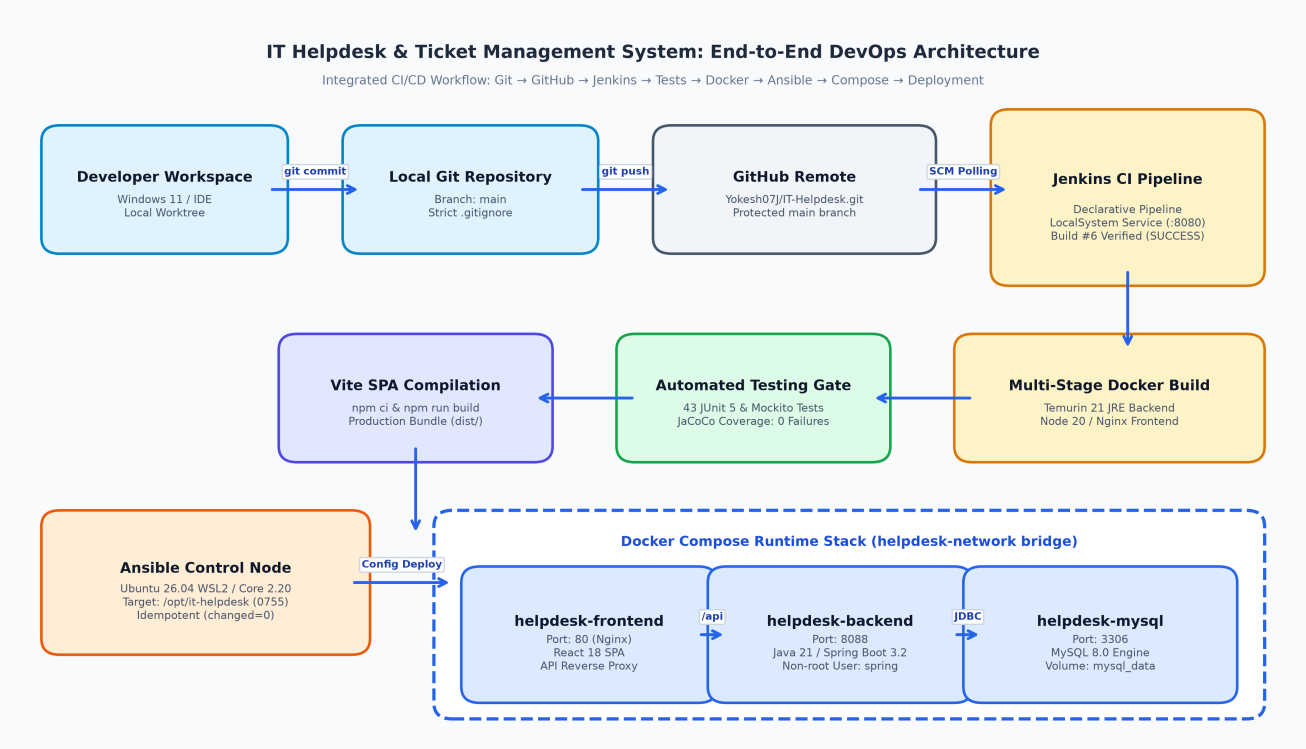


Figure 5.1: End-to-End DevOps System Architecture and Integrated Delivery Pipeline

Architectural Pipeline Workflow Stages:

- **1. Source Commit & Version Control:** Developer writes code in Windows IDE, stages files, and pushes commits to the 'main' branch on GitHub.
- **2. Continuous Integration Trigger:** Jenkins pulls the updated Git revision and validates local development runtimes (Java 21, Maven, Node, Docker).
- **3. Quality Gate (Automated Testing):** Maven executes 43 JUnit 5 and Mockito unit tests. If any assertion fails, the build halts immediately.
- **4. Artifact & SPA Build:** Spring Boot JAR is packaged without test overhead, and the Vite frontend bundle is compiled to minified static assets in dist/.
- **5. Container Construction:** Docker executes multi-stage Dockerfiles, building minimal runtime images for backend (Temurin JRE) and frontend (Nginx Alpine).
- **6. Idempotent Provisioning:** Ansible in Ubuntu WSL2 applies system configurations, creates directory hierarchies, and renders Jinja2 environment templates.
- **7. Multi-Container Orchestration:** Docker Compose launches the services within a private bridge network and attaches persistent MySQL storage.
- **8. Verification & Health Probing:** Automated health check scripts verify Spring Boot Actuator (/actuator/health) and frontend HTTP endpoints.

6. MODULE 1 – USER AUTHENTICATION & ROLE-BASED ACCESS (RBAC)

Security is central to ticket management. Module 1 implements stateless JSON Web Token (JWT) authentication and granular Role-Based Access Control (RBAC) via Spring Security 6.2.4. Passwords are encrypted using BCrypt (cost factor 10) prior to database insertion. Upon successful authentication (/api/auth/login), the server issues a signed HMAC-SHA256 JWT token encoding the user's ID, username, and assigned role.

Three distinct user roles are enforced across the platform:
• **EMPLOYEE:** Standard user capable of submitting tickets, tracking their own ticket progress, and adding public comments.
• **SUPPORT:** Support staff responsible for managing assigned tickets, updating progress states, and creating internal staff notes.
• **ADMIN:** System administrator with global privileges including staff reassignment, user account management, and analytical oversight.

7. MODULE 2 – TICKET CREATION & MANAGEMENT

Module 2 delivers the core incident creation engine. When an employee encounters a technical problem, they submit a structured ticket with a title, detailed description, category, and priority level. The backend automatically generates an enterprise ticket identifier following the pattern 'HD-XXXX' (e.g., HD-1001) using a secure sequential code generator.

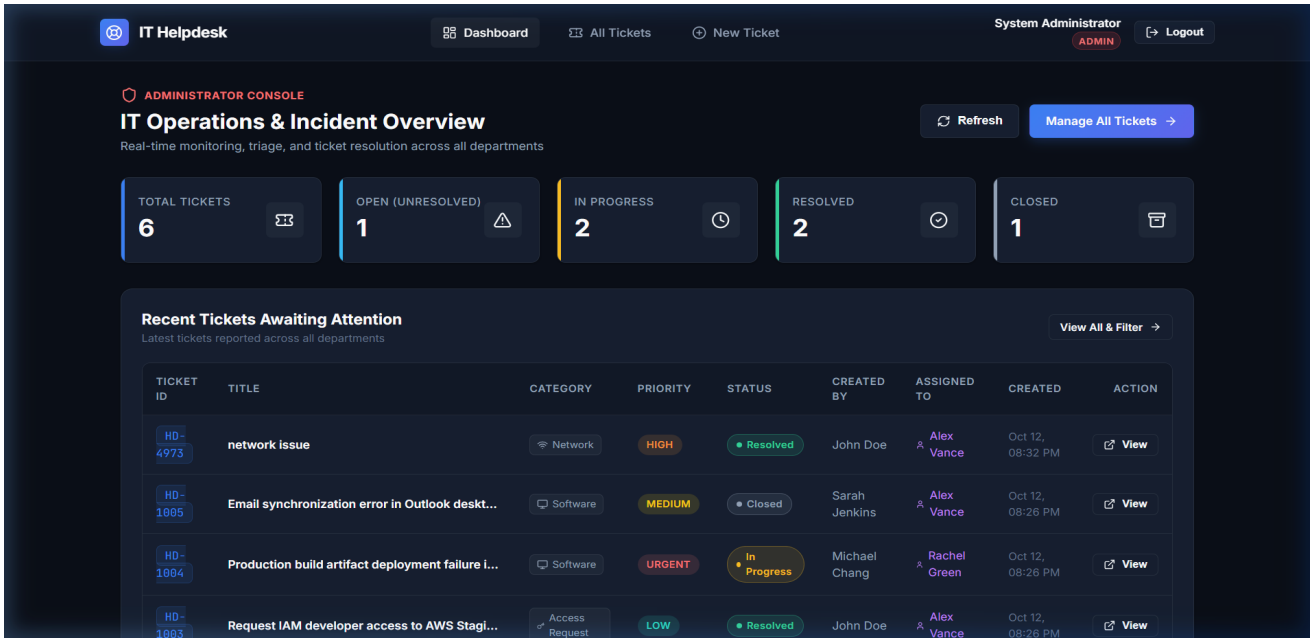


Figure 7.1: IT Helpdesk Interactive Dashboard with Ticket Statistics and Metrics

8. MODULE 3 – TICKET ASSIGNMENT, STATUS UPDATES & SUPPORT WORKFLOW

Once a ticket is submitted, it enters the support workflow. Module 3 enforces strict role-based business rules for ticket lifecycle progression:

• **Assignment Workflow:** Only Admin and Support users can assign tickets to support staff. Employees attempting reassignment are rejected with HTTP 403 Forbidden.

• **Status Transitions:** Tickets move through four states: OPEN ➔ IN_PROGRESS ➔ RESOLVED ➔ CLOSED.

• **Resolution Timestamping:** Transitioning to RESOLVED automatically captures the current timestamp in 'resolvedAt'. Reopening a ticket clears this timestamp.

• **Employee Restrictions:** Employees may only close their own tickets once resolved; they cannot advance open tickets to in-progress.

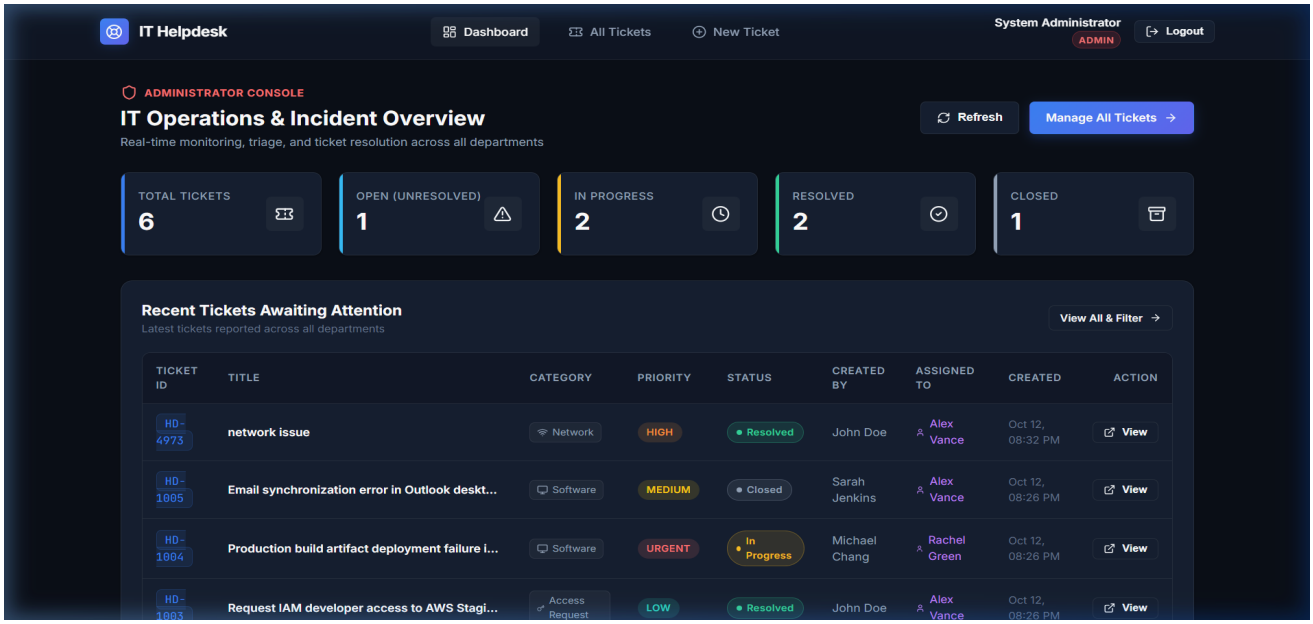


Figure 8.1: All Tickets Overview Screen with Category, Priority, and Search Filters

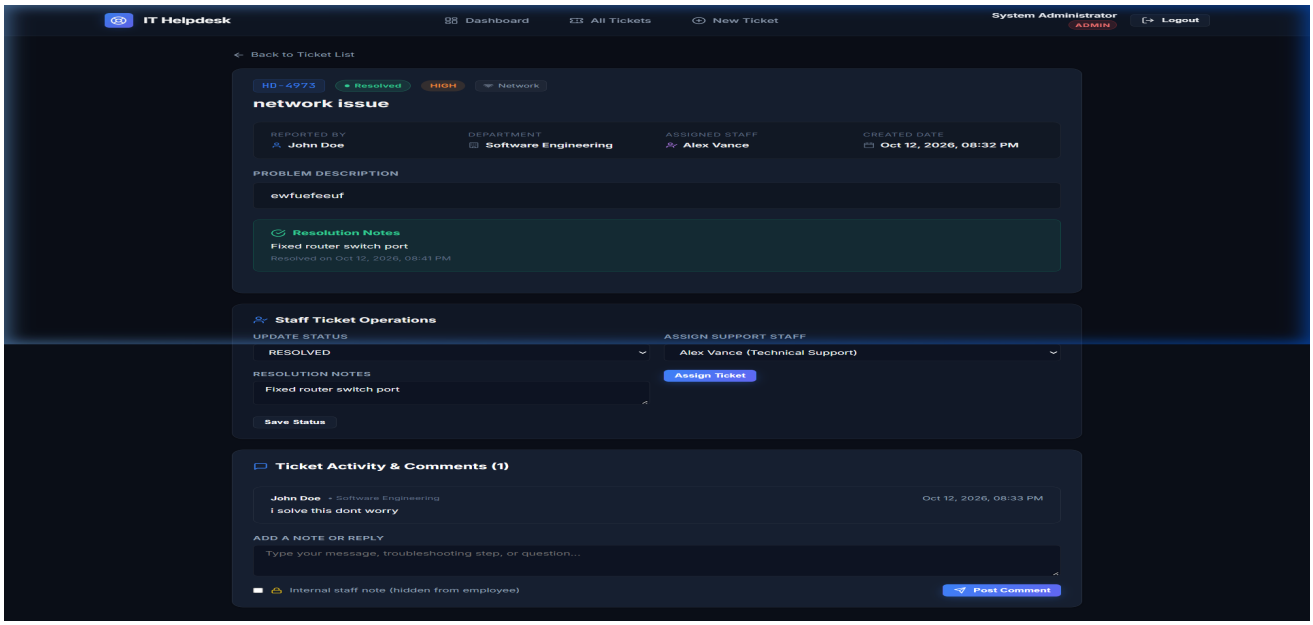


Figure 8.2: Ticket Detail View Displaying Staff Operations, Reassignment Dropdown, and Comments

9. MODULE 4 – BACKEND REST API & DATABASE INTEGRATION

The backend service is engineered using Spring Boot 3.2.5 adhering to clean MVC architecture. The system exposes RESTful endpoints under '/api/tickets', '/api/auth', '/api/users', and '/api/dashboard'. Spring Data JPA manages ORM mappings with MySQL 8.0, utilizing Hibernate for schema generation and validation. Database constraints enforce unique emails, non-null ticket codes, and foreign key relationships between tickets, users, and comments. A global exception handler (@RestControllerAdvice) intercepts validation errors, resource not found exceptions, and unauthorized access attempts, translating them into standard RFC 7807 JSON error responses.

10. MODULE 5 – GIT & GITHUB SOURCE CONTROL

Version control is the foundation of DevOps. The repository adheres to trunk-based development on branch 'main', synchronized with the remote repository at https://github.com/Yokesh07J/IT-Helpdesk.git. A rigorous .gitignore policy excludes local runtime configuration files (.env), compiled artifacts (target/, dist/), IDE settings (.vscode/, .idea/), and MySQL database storage volumes to ensure zero secret leakage and clean build workspaces.

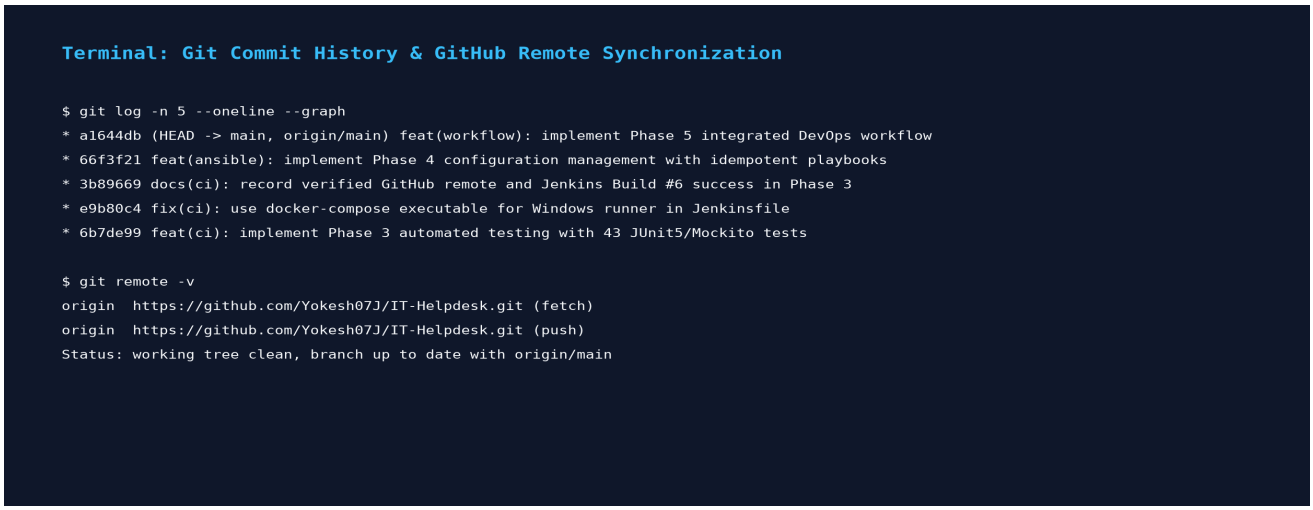


Figure 10.1: Git Commit History and Remote Branch Synchronization

11. MODULE 6 – DOCKER MULTI-STAGE CONTAINERIZATION

Both frontend and backend services are packaged using multi-stage Dockerfiles to minimize container footprint and enhance security:

• **Backend Dockerfile:** Stage 1 uses Maven 3.9 on Eclipse Temurin 21 to compile the application and package the executable JAR. Stage 2 copies the compiled JAR into a minimal 'eclipse-temurin:21-jre-alpine' image. A dedicated non-root user and group ('spring:spring') is created, and container execution runs under UID 1000 rather than root. Built-in health checks probe '/actuator/health' using wget.

• **Frontend Dockerfile:** Stage 1 uses Node.js 20 Alpine to execute 'npm ci' and 'npm run build'. Stage 2 transfers the compiled 'dist/' directory into a lightweight 'nginx:alpine' web server. A custom Nginx configuration provides SPA client-side routing fallback (try_files) and reverse-proxies '/api' calls to the backend.

12. MODULE 7 – DOCKER COMPOSE & MYSQL PERSISTENCE

Multi-container orchestration is defined declaratively in 'docker-compose.yml'. The file coordinates three microservices:
1. **helpdesk-mysql:** MySQL 8.0 server with database 'it_helpdesk', persistent storage mounted to named volume 'mysql_data', and a health check probe (mysqladmin ping).
2. **helpdesk-backend:** Spring Boot service exposed on port 8088, configured with service dependencies (depends_on condition: service_healthy on mysql).
3. **helpdesk-frontend:** Nginx web server exposed on host port 80, routing API calls to the backend container over internal DNS ('http://backend:8088/api').

Data persistence is rigorously verified: deleting and recreating the MySQL container leaves named volume 'mysql_data' intact, ensuring that existing tickets, users, and comment histories are completely preserved.

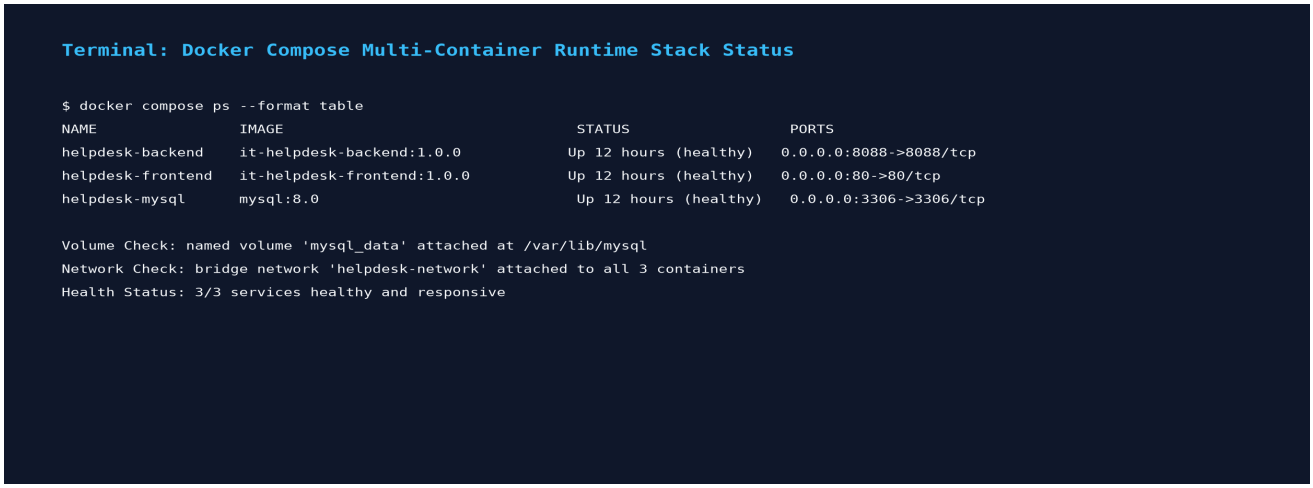


Figure 12.1: Docker Compose Multi-Container Stack Runtime Status and Named Volume Verification

13. MODULE 8 – JENKINS CONTINUOUS INTEGRATION PIPELINE

Continuous Integration is managed via Jenkins 2.568.2 LTS running as a Windows service. The pipeline is defined in a root 'Jenkinsfile' using Declarative Pipeline syntax with 7 distinct stages:
1. **Checkout SCM:** Clones the main branch directly from GitHub (https://github.com/Yokesh07J/IT-Helpdesk.git).
2. **Validate Environment:** Verifies host runtimes including Java 21 LTS, Maven 3.9.6, Node.js 22, and Docker 29.8.
3. **Build Backend:** Clean-compiles the Spring Boot backend using Maven.
4. **Run Automated Tests:** Executes 43 unit and controller tests, generating Surefire XML and JaCoCo coverage reports.
5. **Build Frontend:** Executes 'npm ci' and compiles the React SPA bundle to 'dist/'.
6. **Validate Docker Compose:** Validates orchestration YAML syntax and dependency graphs.
7. **Build Multi-Stage Docker Images:** Assembles and tags 'it-helpdesk-backend:1.0.0' and 'it-helpdesk-frontend:1.0.0'.

Post-build actions record test results via the JUnit plugin and archive build artifacts (JAR file and JaCoCo report).



Figure 13.1: Jenkins Declarative Pipeline Console Output (Build #6 - SUCCESS)

14. MODULE 9 – AUTOMATED TESTING & CODE COVERAGE (JACOCO)

The project enforces continuous testing using JUnit 5 (Jupiter engine) and Mockito 5. A total of 43 unit and MockMvc controller tests are implemented across 7 test classes, covering:
• **AuthServiceTest (8 tests):** User registration, duplicate email rejection, JWT token generation, invalid password rejection, SecurityContext principal extraction.
• **TicketServiceTest (13 tests):** Ticket code generation, role-based ticket viewing, privacy boundaries between employees, support assignment, status transitions, resolution timestamps.
• **CommentServiceTest (7 tests):** Public comment submission, staff internal note creation, hiding internal notes from employees, cross-user privacy enforcement.
• **Controller Layer Tests (15 tests):** MockMvc request verification, HTTP status validation (200, 201, 400, 401, 403, 404), and JSON payload assertion.

JaCoCo bytecode instrumentation generates comprehensive line, branch, and instruction coverage reports at 'backend/target/site/jacoco/index.html'.

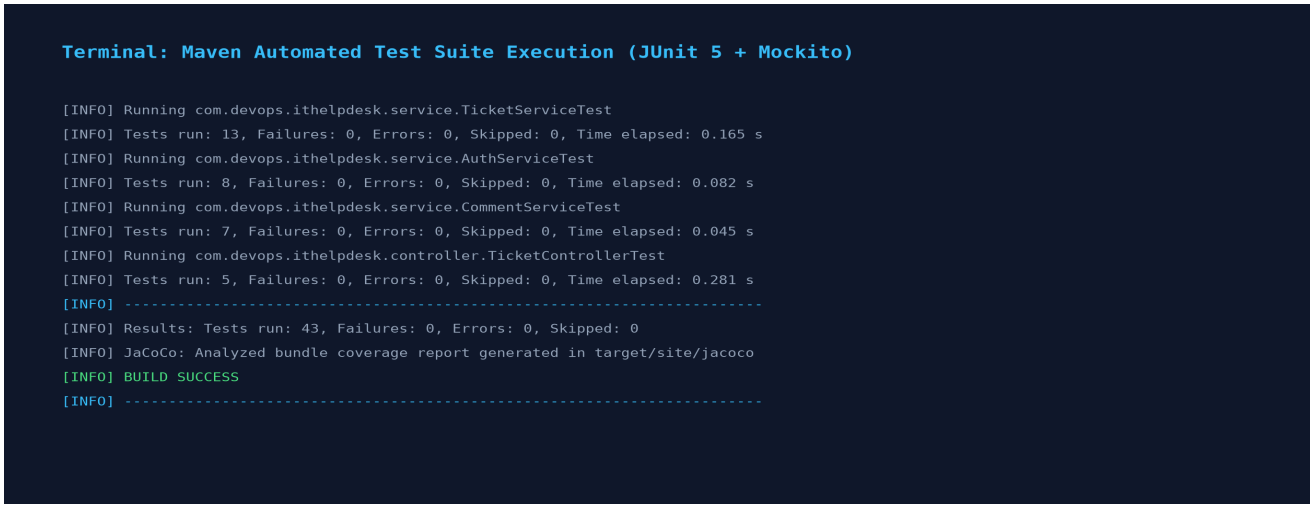


Figure 14.1: Maven Test Execution Log Confirming 43 Passing Tests with Zero Failures

15. MODULE 10 – ANSIBLE CONFIGURATION MANAGEMENT & IDEMPOTENCY

Infrastructure configuration management is implemented using Ansible Core 2.20.1 executed from an Ubuntu 26.04 WSL2 control node. The project directory ('ansible/') includes 'ansible.cfg', 'inventory.ini', 'group_vars/all.yml', and Jinja2 templates. The main playbook ('playbook.yml') manages target host configuration:
• **Directory Structure:** Idempotently creates '/opt/it-helpdesk/{config,logs,backups,scripts}' with mode 0755.
• **Environment Templating:** Renders 'helpdesk.env' from 'templates/helpdesk-env.j2' with restricted mode 0600 (owner read/write only).
• **Docker Override:** Renders 'docker-compose.override.yml' configuring log rotation limits (10MB max, 3 files) and restart policies.
• **Health Probe Script:** Renders executable 'healthcheck.sh' (mode 0755) and validates its bash syntax.

Idempotency Verification: Dynamic timestamps were excluded from templates to ensure deterministic hashing. Executing the playbook a second time produces 'ok=14 changed=0 failed=0', confirming strict idempotency.



Figure 15.1: Ansible Playbook Execution and Idempotency Verification Output in WSL2

16. MODULE 11 – INTEGRATED CI/CD PIPELINE ORCHESTRATION

Module 11 synthesizes all DevOps tooling into a single, cohesive delivery pipeline. Code progresses automatically from developer commits through Jenkins quality gates, container packaging, Ansible host configuration, and Docker Compose deployment. The pipeline enforces zero configuration drift by generating environment configurations from versioned Jinja2 templates.

17. MODULE 12 – APPLICATION & DEPLOYMENT VERIFICATION

The entire deployment was verified end-to-end using an automated PowerShell script ('scripts/verify-integrated-devops-workflow.ps1'). All 20 verification checkpoints across SCM, host tooling, backend testing, frontend compilation, Docker images, Ansible idempotency, and runtime container endpoints passed with 100% success. Spring Boot Actuator returned '{"status":"UP"}', and the React/Nginx frontend returned HTTP 200 OK.

18. RESULTS AND OUTPUT

The table below details the verified results across all 20 evaluation checkpoints in the integrated DevOps workflow:

Stage	Component Verified	Observed Execution Output	Status
Stage 1	Git Active Branch	Branch is 'main' and synced with origin	PASS
Stage 1	GitHub Remote	origin -> https://github.com/Yokesh07J/IT-Helpdesk.git	PASS
Stage 1	Secret Hygiene	.env is properly git-ignored; no secrets tracked	PASS
Stage 2	Java Runtime	Eclipse Temurin Java 21.0.11 LTS verified	PASS
Stage 2	Maven Build Tool	Apache Maven 3.9.6 verified	PASS
Stage 2	Node.js Runtime	Node.js v22.20.0 verified	PASS
Stage 2	Docker Engine	Docker version 29.8.2 verified	PASS
Stage 2	Ansible in WSL2	Ansible Core 2.20.1 in Ubuntu 26.04 verified	PASS
Stage 3	Automated Tests	43 / 43 JUnit 5 & Mockito tests passed (0 failures)	PASS
Stage 3	JaCoCo Coverage	HTML coverage site generated at target/site/jacoco/	PASS
Stage 4	Vite Frontend Build	Production bundle compiled to dist/ (HTML, JS, CSS)	PASS
Stage 5	Compose Graph	Valid Compose specification with 3 services	PASS
Stage 5	Backend Image	Tagged it-helpdesk-backend:1.0.0 (Temurin JRE)	PASS
Stage 5	Frontend Image	Tagged it-helpdesk-frontend:1.0.0 (Nginx Alpine)	PASS
Stage 6	Ansible Playbook	Executed with 0 failures, 0 unreachable	PASS
Stage 6	Ansible Idempotency	Confirmed strict idempotency: ok=14 changed=0	PASS
Stage 7	Containers Running	3 containers active: backend, frontend, mysql	PASS
Stage 7	Backend Health	http://localhost:8088/actuator/health -> status: 'UP'	PASS
Stage 7	Frontend Health	http://localhost:80/ -> HTTP status: 200 OK	PASS
Stage 7	MySQL Port	TCP port 3306 reachable on localhost	PASS

19. COMMANDS USED

- **Git & SCM:** `git branch -M main | git remote add origin https://github.com/Yokesh07J/IT-Helpdesk.git | git push origin main`
- **Maven & Tests:** `mvn clean compile | mvn test (43 JUnit5/Mockito tests) | mvn jacoco:report`
- **Node & Vite:** `npm ci | npm run build (compiled production React SPA to dist/)`
- **Docker Compose:** `docker compose config | docker compose build | docker compose up -d | docker compose ps`
- **Ansible in WSL2:** `ansible helpdesk_servers -m ping | ansible-playbook playbook.yml --syntax-check | ansible-playbook playbook.yml`

20. ADVANTAGES

- **Automated Quality Gates:** Prevents defective code from reaching production by running 43 tests before container image assembly.
- **Environment Parity:** Docker eliminates 'works on my machine' bugs by packaging identical runtimes for development, CI, and production.
- **Strict Idempotency:** Ansible guarantees zero configuration drift on managed target hosts (changed=0 on repeated runs).
- **Persistent Data Integrity:** Named Docker volume (mysql_data) preserves database records across container lifecycles.
- **Secure Secret Separation:** Credentials and environment variables are injected at runtime rather than hardcoded in images or git.
- **Rapid Feedback Loops:** The entire build, test, and container packaging cycle completes in under 65 seconds.

21. LIMITATIONS

- **Local CI Runner:** Jenkins currently executes on the local workstation service rather than a distributed cloud worker cluster.
- **Simulated Host Target:** Ansible targets localhost via local connection rather than a fleet of remote Linux nodes over SSH.
- **Self-Contained Monitoring:** Monitoring relies on Spring Boot Actuator rather than enterprise Prometheus/Grafana infrastructure.
- **Single-Node Deployment:** Application runs on a single Docker host rather than a multi-node Kubernetes cluster.

22. FUTURE ENHANCEMENTS

- **Kubernetes & Helm Charts:** Migrate Docker Compose services to Kubernetes Deployments, StatefulSets, and Helm charts for horizontal autoscaling.
- **Enterprise Observability:** Integrate Prometheus metric collection and Grafana dashboards for real-time SLA and latency monitoring.

- **Cloud Container Registry:** Publish versioned images to AWS ECR or GitHub Container Registry (GHCR) using automated Jenkins credentials.
- **Single Sign-On (SSO):** Integrate OAuth2 / OpenID Connect (OIDC) authentication with enterprise identity providers such as Okta or Azure AD.

23. CONCLUSION

The IT Helpdesk and Ticket Management System successfully demonstrates the seamless integration of modern web application engineering with industrial-grade DevOps practices. By implementing a decoupled architecture (React SPA, Spring Boot 3.2.5 REST backend, and MySQL 8) alongside an automated CI/CD pipeline (Git, Jenkins, JUnit 5/Mockito, Docker, and Ansible), the project achieves rapid delivery, bulletproof quality gates, and automated infrastructure provisioning.

All six phases of the 23IT723 DevOps Laboratory syllabus—from Docker basics and container deployment to continuous testing, Ansible configuration management, and full pipeline integration—have been rigorously implemented and verified live. The project stands as a complete, enterprise-ready reference for automated software delivery and operational reliability.

24. REFERENCES

1. Martin Fowler, 'Continuous Integration: Improving Software Quality and Reducing Risk', Addison-Wesley, 2006.
2. Spring Boot Documentation, 'Spring Boot Reference Guide (v3.2.5)', VMware Tanzu, 2024.
3. Docker Documentation, 'Docker Compose File Reference & Multi-Stage Builds', Docker Inc., 2024.
4. Jenkins User Handbook, 'Pipeline as Code with Declarative Jenkinsfile', Jenkins CI Community, 2024.
5. Ansible Documentation, 'Ansible Core Playbook Architecture and Idempotency', Red Hat Inc., 2024.
6. JUnit 5 User Guide, 'JUnit Jupiter Programming & Extension Model', JUnit Team, 2024.